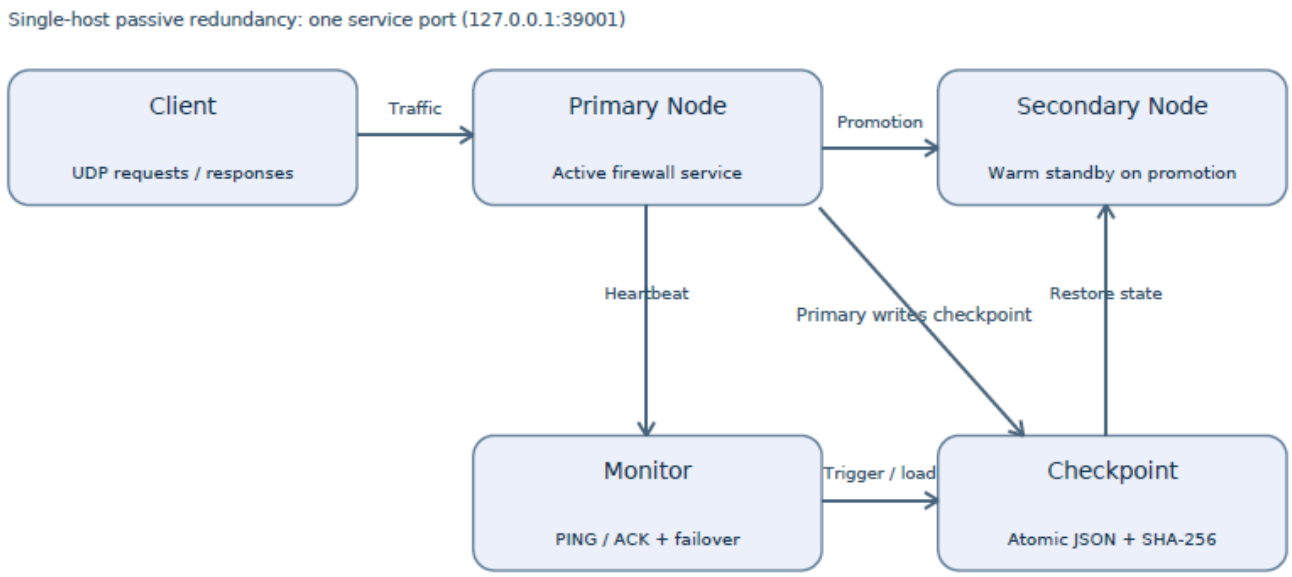
Network Sentinel | Part 2

Fault-Tolerant Firewall Cluster - Submission Report

a. Team members

Jesse Martin

Software architectural diagram



The monitor supervises the primary and probes it over UDP. The primary checkpoints dynamic state before acknowledging state-changing requests. When a supervised primary exits and the heartbeat threshold is reached, the monitor launches the standby, which validates and restores the checkpoint before binding the same UDP service port. Client requests must be retried during the brief handoff.

c. Redundancy justification

Selected model: Option A, passive redundancy (warm standby with checkpointing). This matches a lightweight, single-host Python prototype: only one node serves client requests at a time, while a persistent checkpoint supplies the backup with committed state. Dynamic state includes total inspected requests, allowed/blocked counters, and an updated blacklist.

Trade-offs: Passive replication has lower ongoing synchronization traffic than an active hot spare, but each state-changing request incurs local checkpoint I/O. It is simpler to coordinate because only one service owns the client port. Active redundancy could shorten recovery time but would require simultaneous replicas, state ordering, conflict handling, and robust leader election to avoid split-brain. Passive failover has a higher Mean Time to Repair (MTTR): the monitor must first detect the failure, launch the standby, validate state, and wait for it to answer.

Timing: Heartbeat interval T = 1 s, acknowledgment timeout W = 2 s, threshold N = 3 misses. Sequential timeout waits yield approximately 6 s from the first unanswered probe to threshold detection, plus startup/recovery time. Actual MTTR varies with crash timing and operating-system scheduling. A process still alive but missing ACKs is not replaced, prioritizing avoidance of duplicate primaries over availability.

Execution and adversarial assessment

d. Execution and setup instructions

Requirements: Python 3.10+; standard library only. No compilation or pip installation. Extract the project ZIP and open two terminals in its folder. On Windows, use py instead of python if needed.

Terminal 1 - launch the monitor and primary:

```
cd network_sentinel_part2 python monitor.py --failure-probability 0
```

Terminal 2 - create and inspect persistent state:

```
python client.py inspect --ip 8.8.8.8 python client.py inspect --ip 192.168.1.100 python client.py block --ip 8.8.8.8 python client.py inspect --ip 8.8.8.8 python client.py status
```

The primary writes checkpoint.json after each state-changing request. The final STATUS response shows counters and the dynamic blacklist.

Trigger stochastic crash and automatic failover: Stop the monitor using Ctrl+C, then restart it in Terminal 1: `python monitor.py --failure-probability 0.20`

Wait for a random crash, then three missed heartbeats. Watch for timestamped [Crash Detected], [Failover Triggered], and [Service Restored] messages. Run python client.py status again to verify that the backup retains the counters and blacklist. Incident messages are also written to monitor.log.

Run Task 4 audit (Scenario 2): With a valid checkpoint already created, run:

```
python checkpoint_audit.py
```

The script modifies a temporary checkpoint copy, launches an isolated backup on UDP port 39099, and checks that the backup refuses to load the invalid state. The original checkpoint is not modified.

Optional split-brain probe:

```
python monitor.py --failure-probability 0 --drop-acks
```

This simulates dropped heartbeat acknowledgments. The monitor should log failover suppression while the supervised primary process remains alive. Stop other monitor instances before running this test.

e. Adversarial audit summary

The team selected Scenario 2, tampered checkpoint/failover synchronization. The auditor alters the blocked-request counter in a temporary copy of the persisted JSON checkpoint without recalculating its SHA-256 checksum. The audit then attempts to start a standby from the tampered file. The intended and previously observed outcome is rejection of the checkpoint with a nonzero exit status, rather than promotion using inconsistent firewall state. This is a fail-closed result: integrity is prioritized, although service restoration is unavailable until a valid checkpoint is provided.

The checksum detects accidental corruption and simple edits, but is not authentication: an adversary with write access could change both the data and checksum. Stronger defenses would include authenticated checkpoints (for example, HMAC), restricted filesystem permissions, checkpoint versioning, and rollback to a known-good copy. The optional ACK-loss scenario tests split-brain safety; because the monitor checks whether its supervised primary process is still alive, it suppresses promotion on heartbeat loss alone. This protects the single-host prototype, but does not solve distributed split-brain or host-wide failures.

f. Contributions

See the accompanying CONTRIBUTIONS.md file.